

基于 DPDK 的二层转发技术分析

赵冲 胡炜 阳春

数据平面开发套件 (DPDK) 软件设计是由一组用户层面的库文件和相关驱动组成,可加速在所有主要 CPU 架构上运行的数据包处理工作负载。由 Intel 大约 10 年前创建。从企业数据中心到公共云的环境,尤其是在电信网络中,DPDK 在推动高性能通用 CPU 的使用方面都起了重大作用。本文通过对 DPDK 架构、技术特点及二层转发实例框架设计进行研究分析,探讨 DPDK 报文吞吐相较于 Linux 内核处理的性能优势。

Linux内核收发包处理流程分析

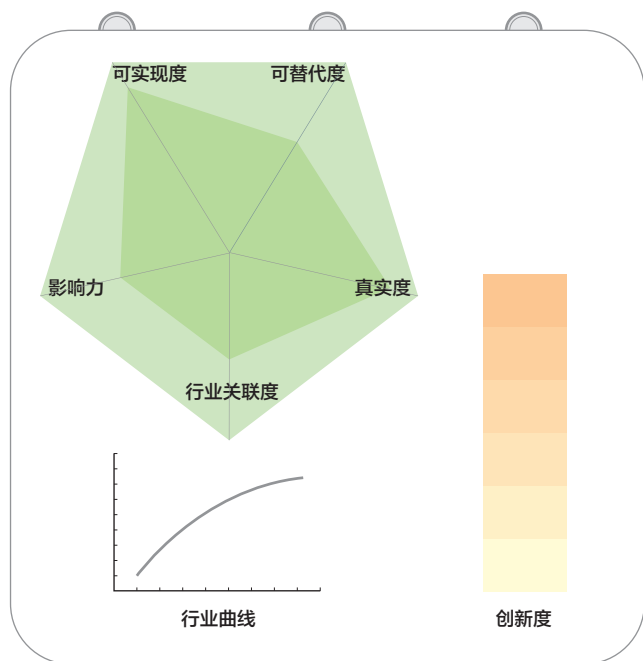
当网卡收到报文后放入内存,同时触发中断通知内核有报文到达(一个报文,一个中断),系统分配 sk_buff,将报文 copy 到这块 sk_buff 中,然后交由协议栈处理,经过协议栈处理,将报文上送用户态应用程序处理;当有报文需要发送时,应用层通过系统调用经过内核协议栈处理后,将发送数据报文拷贝至内核层创建的 sk_buff 中,并触发软中断将报文发送。

从上面的简单描述可以看出,原生态协议栈处理越多报文就消耗越多资源(中断,内存等)。一个报文一个中断,中断打断 CPU 并且涉及多次上下文切换。每个报文的收发都需要系统分配对应 sk_buff 结构,且内核为兼容更多协议导致该结构非常复杂与庞大。这些都增加了系统的资源消耗与处理延迟问题。

DPDK架构设计及核心组件

DPDK 全称数据面开发套件(又称数据平面开发工具集),为 Intel 处理器框架下用户层面数据报文高效快速处理提供了应用库接口函数和驱动实现。DPDK 对传统 Linux 技术做了一定的优化,特别之处在于 hugepage、uio、zero copy、cpu affinity 等技术。由于 DPDK 处于应用层面,与底层网卡驱动相关都通过 UIO(用户层驱动、轮询、0 拷贝)机制在用户态下实现,传入网络数据包过渡到用户空间,绕过内核协议栈处理,进而消除了内存复制的开销,大幅度提高网络数据的转发性能。DPDK 有以下技术特点:

(1) 控制层和数据层分离,将数据报文处理、内存相关管理、处理器调度等任务转移到用户空间去完成,而系统内核仅仅处理部分控制指令。这样就不存在上述所说的系统频繁中断、上下文两态切换、系统调用与调度等等问题。



英特尔开发的数据面开发套件 DPDK,通过对网络应用高性能的实现,逐渐获得业界的认可。本文从 Linux 内核原生网络协议栈处理过程中存在的瓶颈入手,分析 DPDK 的架构、组件设计构成,通过 DPDK 提供的库函数,并结合 DPDK 的特性,基于 DPDK 的二层转发 l2fwd 程序,通过网卡端口建立多个 Tx/Rx Queue 并将其绑定在不同 CPU 逻辑核上实现对数据报文的处理。通过实验结果对比,相较于 Linux 内核原生数据转发实现方式,具有高性能的包转发能力。

赵冲 胡炜 阳春
中国电子科技集团公司第三十研究所



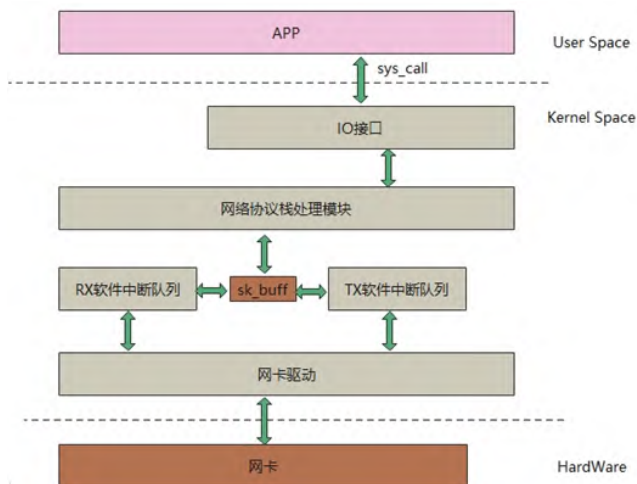


图 1 内核协议栈收发包简要流程

(2) 多核技术替代多线程，通过 CPU 亲和性将线程与 CPU 逻辑核进行对应绑定，减少多核间彼此调度频繁切换。

(3) NUMA 亲和性。NUMA 系统中，采用 CPU 核所使用的 NUMA 节点的系列内存，避免核与核间跨内存进行数据访问。

(4) 使用大页内存减少 cache-miss。

(5) 采用无锁技术实现资源无竞争读写。

架构设计

如图 2 所示 DPDK 架构图，系统底层采用 UIO 技术，将硬件资源（MMIO，IO，interrupt）映射到用户态，实现用户态和内核态的 zero 拷贝数据包交互，将数据包的控制权交给应用层的应用程序进行处理。DPDK 提供各种功能库：MEMORY、RING、TIMER、MBUF、PDM、MEMPOOL、LPM 等，以及多种平台模块。

核心组件分析

DPDK 核心组件主要有 EAL、PMD、流分类、环队列、MBUF 缓冲区管理、定时器管理等，如图 2 所示。其设计核心思想：

采用用户态轮询驱动去除中断影响，充分利用网卡性能。
避免内核态和用户态之间数据报文拷贝；
用户层面应用崩溃，对操作系统稳定性不构成影响；
提供 2M 和 1G 的巨页内存，减少了 TLB Miss；
高效的 mbuf 管理，提高报文组织的灵活性；
消息队列之间无锁化；
采用向量指令实现，大幅降低内存等待开销。

DPDK L2-forward 程序设计与实现

在操作系统中，二层是链路层，是以太网所在的层，识别的是设备端口的 MAC 地址。网卡驱动在二层上的作用就是根据设定的目的端口，转发报文到目的端口。传统方式接收报文时，当网卡接收到数据报文后会产生异步信号的硬件

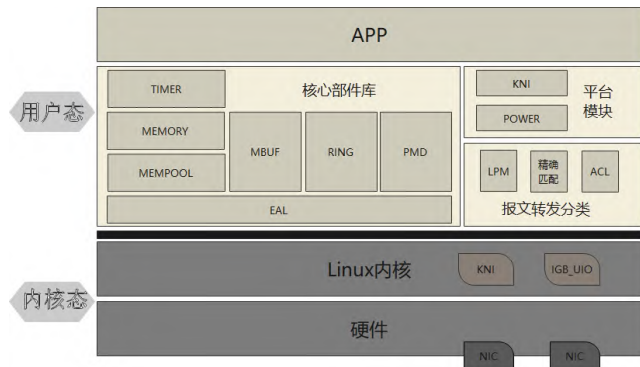


图 2 DPDK 框架图

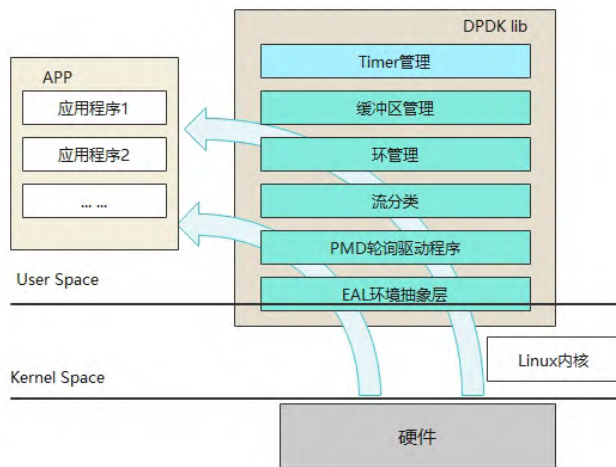


图 3 DPDK 核心组件

中断，进而报文会通过协议栈，最后达到应用，这个过程需要内核协议栈的处理。DPDK 作为用户态驱动，主要的目的也就是不需要让报文经过操作系统协议栈而能实现快速的转发功能，和传统报文接收不同，当应用层需要接收来自网卡报文时，应用层通过 while 死循环方式，调用 burst 接口轮询接收报文，将内核旁路提高内存访问效率。

l2fwd 程序主流程

l2fwd 启动主流程如图 4。其实现过程如下：

(1) 程序入口调用 rte_eal_init (EAL 环境适配初始化) 接口，该接口为上层 DPDK APP 提供一个通用接口，隐藏了与底层设备交互相关细节。主要完成 DPDK 加载与启动，将执行单元分配给特定核心的机制以及创建其执行实例，进而完成系统内存的预留及中断处理、报警功能、巨页分配、从线程初始化等功能。rte_mempool 私有数据结构由 rte_mempool_create 创建内存并完成初始化，主要用于网卡数据包的收发提供内存。

(2) 通过 l2fwd_parse_args 接口解析逻辑 Core、Per CPU 通道、PortMask 等数量，设置并初始化配置二层转发端口信息，为每个端口分配到相应的逻辑内核。

(3) rte_eal_remote_launch 接口通过向每个核心运行的线程发送信号，为每个逻辑 core 完成任务分发。

最后，master lcore 检查各个端口连接状态并在每个逻辑内核上启动收发包从线程，开始数据转发处理。

I2fwd 应用程序为 RX_PORT 上接收的每个数据包执行二层转发。目标端口是启用的端口掩码的相邻端口。即：如果启用前四个端口（端口掩码 0xf，每个端口用一个比特位表示，启动 4 个就是 4 个比特位置 1），端口 1 和 2 相互转发，端口 3 和 4 相互转发。DPDK 收发包是核心模块，从网卡收到包到驱动把包拷贝到系统内存，以及对这块数据包的内存管理，整个转发的实现过程，可以说是将 mbuf、mempool、dma 控制器、描述符队列关联的过程。

报文接收流程

当网卡接收到报文时将会产生硬件中断，通知 DMA 控制器接收报文。DMA 控制器会从网卡 Rx Queue 中将报文拷贝至硬件描述符空间指向的地址位置，也就是 mbuf，最终 DMA 控制器将报文从网卡接收队列中拷贝到 mbuf，应用层通过 while 循环，轮询调用 rte_eth_rx_burst 接口将 mbuf 中获取报文。

应用层调用 rte_eth_rx_burst 接口接收报文，函数内部会调用 PMD 用户态驱动接收报文接口 eth_igb_rcv_pkts。具体实现简要流程：

从描述符中获取报文返回给应用层。

应用层根据最后一次获取报文的位置，从描述符队列中找到等待被接收的描述符。根据描述符中 status_error 是否含有 dd 标记（该标记为 DMA 控制器将报文写入 mbuf

完成标识），应用层获取完报文后清除 dd 标记。找到对应 mbuf 空间位置信息后，根据描述符结构中保存的信息，完成 mbuf 结构信息的填充后传递给应用层。

从内存池获取新的 mbuf 通知 DMA 控制器

应用层从软件队列中获取 mbuf 后，从内存池中申请一个 mbuf 空间，将该地址信息放到描述符队列中。此时对描述符中 dd 标记已清零，当有新的报文达到时，DMA 控制器将报文写入新的 mbuf 中。

报文发送流程

当应用层需要发送报文时，调用 rte_eth_tx_burst 接口将报文写到软件 mbuf 空间中。同时将 mbuf 地址信息写入硬件发送空间，也就是描述符空间。DMA 控制器读取描述符空间，从 mbuf 中获取报文，然后通过网卡发送出去。

应用层调用 rte_eth_tx_burst 接口发送报文，函数内部会调用 PMD 用户态驱动 eth_igb_xmit_pkts 接口。实现流程：

将应用层报文写入软件队列中

当应用层发送报文时，根据队列最后一次发送的位置信息，eth_igb_xmit_pkts 内部会将应用层报文写入软件队列相应位置。发送的报文可能需要分片处理，其原则为每个分

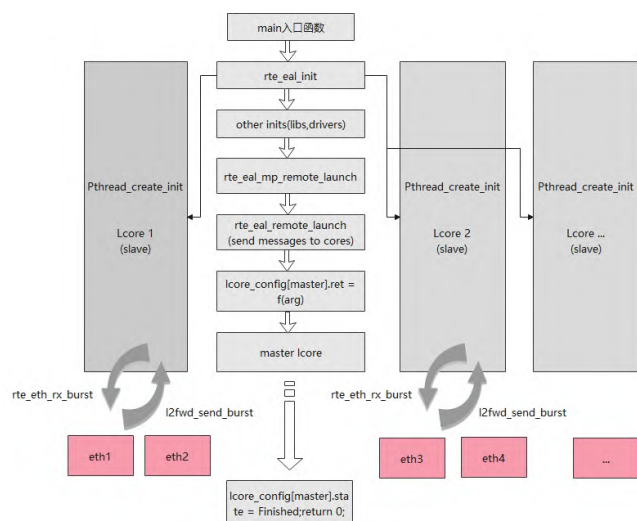


图4 DPK I2fwd 函数流程图

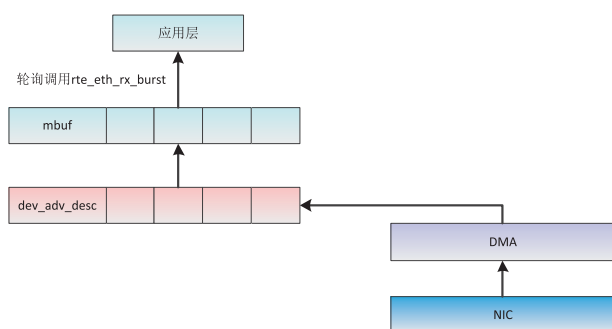


图5 报文接收流程

```
static void
l2fwd_main_loop(void)
{
    ...
    //获取自己的lcore_id
    lcore_id = rte_lcore_id();
    qconf = &lcore_queue_conf[lcore_id]
    ...
    while(!force_quit){
        cur_tsc = rte_rdtsc();
        //计算时间片
        diff_tsc = cur_tsc - prev_tsc;
        if (unlikely(diff_tsc > drain_tsc)) {
            for (i = 0; i < qconf->n_rx_port; i++) {
                portid = l2fwd_dst_ports[qconf->rx_port_list[i]];
                buffer = tx_buffer[portid];
                sent = rte_eth_tx_buffer_flush(portid, 0, buffer);
                if (sent)
                    port_statistics[portid].tx += sent;
            }
        }
        ...
        for (i = 0; i < qconf->n_rx_port; i++) {
            portid = qconf->rx_port_list[i];
            nb_rx = rte_eth_rx_burst((uint8_t) portid, 0,
                pkts_burst, MAX_PKT_BURST);

            //计数，收到的报文数
            port_statistics[portid].rx += nb_rx;
            for (j = 0; j < nb_rx; j++) {
                m = pkts_burst[j];
                rte_prefetch0(rte_pktmbuf_mtod(m, void *));
                l2fwd_simple_forward(m, portid);
            }
        }
    }
} // end l2fwd_main_loop
```

图6 从线程循环伪代码

片的报文 last_id 指向同一个软件队列元素。

将 mbuf 地址信息保存至描述符中。

对于要发送的每一个 mbuf 报文,都需要将 mbuf 地址信息写入描述符队列。DMA 控制器通过读取寄存器获取描述符队列地址,然后通过网卡发送出去。

I2fwd 从线程实现流程

每个逻辑核收发包线程的实现转发性能有着直接的关联。I2fwd 通过调用 rte_eal_mp_remote_launch 接口在每个使能的 lcore 上初始化将要运行的函数,该函数调用 I2fwd_lanuch_one_lcore 完成任务分发,并在每一个收发包线程上执行从线程。收发包线程在循环体内依次在当前 lcore 绑定的端口上进行收包,然后对收到的每个报文调用 I2fwd_simple_forward 进行转发处理。从线程收发实现伪代码如图 2 示。

图 6 为从线程 I2fwd_main_loop 实现伪代码。diff_tsc 为记录当前 tsc 时间减去上一次记录的时间得到了时间差,当延时大于 100us 的时候,遍历当前 lcore 上绑定的每一个端口,调用 rte_eth_tx_buffer_flush 接口立刻发出 buffer 中的报文,然后增加发包统计。没有到发送时间片,每个逻辑核在任务分发后会执行如上循环读接收队列里的报文,直到 I2fwd 收到 SIGINT、SIGTERM 信号时收发包线程检测后退出循环。

二层转发性能实测与评估

为评估 Linux 内核桥转发、DPDK I2fwd 程序转发吞吐率性能,本文采用 XINERTEL 网络测试仪(基于 RFC2544 测试标准),选用 Loongson 3A2000 处理器、Intel82580 千兆网卡,运行 Loongnix 操作系统, Linux 内核版本为 3.10。分别采用 Linux 桥转发、D-I2fwd 程序对包长分别为 64/256/1280 Byte 进行数据转发测试。

Kernel 桥转发测试

(1) brctl 命令行操作完成以太网口 eth1、eth2 网桥配置。

(2) 命令行“sysctl -w net.ipv4.ip_forward=1”开启 ipv4 的转发功能。

测试结果如图 1 所示。测试数据包长为 64 Byte 时,转

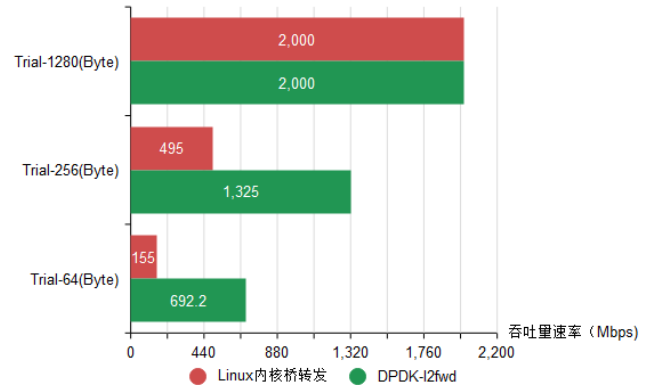


图 7 转发吞吐率测试结果

发数据的吞吐率 155Mbps。提高该吞吐量进行测试丢包率超过 1%，已达性能瓶颈。

D-I2fwd 程序转发测试

配置 DPDK 运行环境,完成大页内存的分配,安装 igb_uio 驱动并通过 dpdk_nic_bind.py 脚本绑定网口后,设置环境变量后运行命令行“I2fwd -c 0x6 -n 2 -- -p 0x3”启动转发程序。

测试结果如图 7 所示。

通过测试结果可看出,测试数据包长为 64 Byte 时,DPDK I2fwd 转发吞吐率相比较 Linux 内核桥转发性能提升 4.5 倍,包长为 256 Byte 时为 2.7 倍,转发性能得到大幅提高。对 I2fwd 的运行参数配置优化调整后,进一步测试发现还存在进一步的提升空间。

技术总结

本文通过对 DPDK 架构设计与 I2fwd 程序进行研究,并进行了 OS 桥、I2fwd 程序转发性能的对比测试。相比原生 Linux 内核处理实现(Native Linux),采用 Intel DPDK 技术后能够大幅提高 IPV4 的数据报文转发性能,可以让用户在移植包处理应用时,获得更好的成本和性能优势。同时可以采用统一的平台对多个应用服务进行部署,如应用、控制和包处理服务,对接用户态 Linux 相关程序,对开发成本降低和硬件通用化实现有很大的帮助。